ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN 1

MA TRẬN VÀ CƠ SỞ CỦA TÍNH TOÁN KHOA HỌC

Môn học:	Toán Ứng Dụng và Thống Kê
Mã môn:	MTH00051
Giảng viên hướng dẫn:	ThS. Võ Nam Thục Đoan
	ThS. Lê Nhựt Nam

Nhóm sinh viên thực hiện

24120442 – Đỗ Đức Duy Thắng
24120437 – Nguyễn Nhật Tân
24120449 – Nguyễn Tấn Thành
24120453 – Trần Hồng Thiên
24120443 – Nguyễn Quang Thắng

TP. HỒ CHÍ MINH, NGÀY 20/04/2025

Mục lục

Thông tin sinh viên và Mức độ đóng góp **1**
0.1 Mức độ đóng góp của thành viên . . . 1
0.2 Mức độ hoàn thành các yêu cầu của đồ án . . . 1

1 Phép khử Gauss và các ứng dụng **3**
1.1 Cơ sở lý thuyết . . . 3
1.1.1 Ma trận tăng cường . . . 3
1.1.2 Các phép biến đổi dòng sơ cấp . . . 3
1.1.3 Dạng bậc thang dòng . . . 3
1.1.4 Partial Pivoting . . . 3
1.1.5 Tính định thức . . . 4
1.1.6 Ma trận nghịch đảo . . . 4
1.1.7 Hạng ma trận . . . 4
1.2 Khử Gauss và Giải hệ phương trình (gaussiant.py) . . . 4
1.3 Tính Định thức (determinant.py) . . . 6
1.4 Ma trận nghịch đảo (inverse.py) . . . 7
1.5 Hạng và Cơ sở không gian (rank_basis.py) . . . 8
1.6 Kiểm chứng kết quả (verify_solution.py) . . . 9

2 Phân rã ma trận và Trực quan hóa với Manim **11**
2.1 Cơ sở lý thuyết: Tầm nhìn từ Chéo hóa đến SVD . . . 11
2.1.1 Chéo hóa ma trận vuông ($A = PDP^{-1}$) . . . 11
2.1.2 Phân rã giá trị suy biến ($A = U\Sigma V^T$) . . . 11
2.2 Cài đặt thuật toán SVD From Scratch . . . 12
2.2.1 Tìm Trị riêng bằng Lặp QR và Modified Gram-Schmidt . . . 12
2.2.2 Xây dựng SVD và Kỹ thuật "Vá lỗ hổng" hệ cơ sở . . . 13
2.2.3 Kiểm chứng và Đánh giá thuật toán . . . 14
2.3 Trực quan hóa hình học với Manim . . . 15
2.4 Ứng dụng SVD . . . 15

3 Giải hệ phương trình và Phân tích hiệu năng **17**
3.1 Phương pháp lặp Gauss-Seidel . . . 17
3.1.1 Cơ sở toán học . . . 17
3.1.2 Chi tiết cài đặt và Xử lý hội tụ . . . 17
3.2 Thực nghiệm đo thời gian và sai số . . . 18
3.2.1 Đánh giá thời gian thực thi . . . 18
3.2.2 Đánh giá độ chính xác . . . 18
3.3 Đánh giá tính ổn định số học (Numerical Stability) . . . 18
3.3.1 Cơ sở lý thuyết về Ma trận Hilbert và SPD . . . 19
3.3.2 Ma trận Hilbert (Ill-conditioned) . . . 19
3.3.3 Ma trận SPD (Well-conditioned) . . . 20
3.3.4 Các kịch bản kiểm thử (Test Cases & Edge Cases) . . . 20
3.3.5 So sánh ma trận Hilbert và ma trận SPD . . . 22
3.4 Tổng kết Phần 3 . . . 23

4 Kết luận và Lời cảm ơn **25**
4.1 Kết luận 25
4.2 Lời cảm ơn 25

Thông tin sinh viên và Mức độ đóng góp

0.1 Mức độ đóng góp của thành viên

Dưới đây là danh sách các thành viên trong nhóm, chi tiết công việc được phân công và mức độ hoàn thành nhiệm vụ của từng cá nhân trong suốt quá trình thực hiện đồ án:

MSSV	Họ và tên	Nhiệm vụ được phân công	Hoàn thành
24120437	Nguyễn Nhật Tân	Cài đặt code thuật toán Phần 2, hỗ trợ kịch bản và xây dựng khung code video Manim, tổng hợp viết báo cáo LaTeX.	100%
24120442	Đỗ Đức Duy Thắng	Tìm hiểu lý thuyết Phần 2, hỗ trợ cài đặt mã nguồn SVD, xây dựng kịch bản và làm video Manim.	100%
24120453	Trần Hồng Thiên	Tìm hiểu lý thuyết, hỗ trợ cài đặt mã nguồn thuật toán Phần 1 và tham gia viết báo cáo.	100%
24120443	Nguyễn Quang Thắng	Tìm hiểu lý thuyết và cài đặt mã nguồn thuật toán Phần 1 (Khử Gauss và Ứng dụng).	100%
24120449	Nguyễn Tấn Thành	Tìm hiểu lý thuyết, thiết lập thực nghiệm, cài đặt code Phần 3 và tham gia viết báo cáo.	100%

0.2 Mức độ hoàn thành các yêu cầu của đồ án

Phần 1: Phép khử Gauss và các ứng dụng

Tiêu chí	Điểm	Hoàn thành
Cài đặt Gauss cơ bản	1.5	100%
Partial Pivoting	0.5	100%
Tính định thức	0.5	100%
Ma trận nghịch đảo	1.0	100%
Hạng và cơ sở	1.0	100%
Kiểm chứng	0.5	100%
Tổng	**5.0**	**100%**

Phần 2: Phân rã ma trận và Manim

Tiêu chí	Điểm	Hoàn thành
Lý thuyết phân rã	0.5	100%
Cài đặt Python	1.5	100%
Chéo hóa ma trận	0.5	100%
Video Manim	2.0	100%
Ứng dụng minh họa	0.5	100%
Tổng	**5.0**	**100%**

Phần 3: Giải hệ và phân tích hiệu năng

Tiêu chí	Điểm	Hoàn thành
Cài đặt phương pháp lặp	1.0	100%
Thực nghiệm thời gian	1.5	100%
Nhận xét và kết luận	1.5	100%
Trình bày Notebook	1.0	100%
Tổng	**5.0**	**100%**

Chương 1. Phép khử Gauss và các ứng dụng

Trong chương này, nhóm trình bày chi tiết về phép khử Gauss — một trong những công cụ nền tảng và quan trọng nhất của đại số tuyến tính số. Xuất phát từ cơ sở lý thuyết về các phép biến đổi dòng sơ cấp, nhóm tiến hành cài đặt hoàn chỉnh thuật toán khử Gauss có chọn phần tử trội (Partial Pivoting) bằng Python từ đầu (from scratch). Từ đó, thuật toán được ứng dụng trực tiếp để giải hệ phương trình tuyến tính, tính định thức, tìm ma trận nghịch đảo, và xác định hạng cùng cơ sở các không gian con của ma trận.

1.1 Cơ sở lý thuyết

Trong đại số tuyến tính, phép khử Gauss là công cụ nền tảng để giải hệ phương trình tuyến tính, tính định thức, tìm ma trận nghịch đảo và xác định hạng ma trận.

1.1.1 Ma trận tăng cường

Ma trận tăng cường của hệ $Ax = b$:

$$[A|b] = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} & b_1 \\ a_{21} & a_{22} & \dots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} & b_m \end{bmatrix}$$

1.1.2 Các phép biến đổi dòng sơ cấp

1. Hoán đổi hai dòng.
2. Nhân một dòng với hằng số khác 0.
3. Cộng một bội số của dòng này vào dòng khác.

1.1.3 Dạng bậc thang dòng

- **REF**: Các dòng 0 nằm dưới, pivot dịch sang phải.
- **RREF**: Pivot bằng 1 và các phần tử khác trong cột pivot bằng 0.

1.1.4 Partial Pivoting

Chọn phần tử lớn nhất theo trị tuyệt đối làm pivot để giảm sai số số học.

1.1.5 Tính định thức

$$\det(A) = (-1)^s \prod_{i=1}^{n} u_{ii}$$

1.1.6 Ma trận nghịch đảo

Biến đổi:

$$[A|I] \to [I|A^{-1}]$$

1.1.7 Hạng ma trận

- $C(A)$: không gian cột
- $R(A)$: không gian dòng
- $N(A)$: không gian nghiệm

1.2 Khử Gauss và Giải hệ phương trình (gaussiant.py)

Thuật toán này giải quyết hệ phương trình tuyến tính $Ax = b$ bằng phương pháp khử Gauss biến đổi ma trận mở rộng về dạng bậc thang.

Phân tích thuật toán chi tiết:

1. **Thiết lập ma trận mở rộng:** Kết hợp ma trận A (kích thước $m \times n$) và vector b (kích thước $m \times 1$) thành ma trận M kích thước $m \times (n + 1)$.
2. **Tìm phần tử trục (Pivoting):** Tại mỗi cột col, thuật toán duyệt từ dòng hiện tại đến dòng cuối cùng để tìm hàng có giá trị tuyệt đối lớn nhất $|M[i][col]|$. Điều này giúp giảm thiểu sai số tính toán khi chia cho số quá nhỏ.
3. **Biến đổi sơ cấp:**
   - **Đổi dòng:** Đưa dòng chứa phần tử trục lên vị trí đang xét.
   - **Chuẩn hóa:** Chia toàn bộ dòng cho giá trị pivot để đưa hệ số tại vị trí trục về 1.
   - **Khử:** Trừ các dòng phía dưới theo công thức $M_i = M_i - factor \times M_{row}$ để tạo các số 0 dưới trục.
4. **Biện luận số nghiệm:**
   - **Vô nghiệm:** Nếu tồn tại dòng mà tất cả hệ số biến bằng 0 nhưng hằng số tự do khác 0 ($abs(M[i][n]) > 1e - 9$).
   - **Nghiệm duy nhất:** Khi hạng của ma trận (số lượng pivot) bằng số ẩn n.

- **Vô số nghiệm:** Khi hạng nhỏ hơn số ẩn. Thuật toán xác định các cột không chứa pivot là biến tự do và biểu diễn các biến phụ thuộc dưới dạng chuỗi biểu thức toán học.

```python
def gaussian_eliminate(A, b):
    m = len(A)
    n = len(A[0])
    M = []
    for i in range(m):
        M.append(A[i] + [b[i]])

    pivots = []
    row = 0
    for col in range(n):
        pivot_row = row
        for i in range(row, m):
            if abs(M[i][col]) > abs(M[pivot_row][col]):
                pivot_row = i
        if abs(M[pivot_row][col]) < 1e-9:
            continue
        M[row], M[pivot_row] = M[pivot_row], M[row]
        pivots.append(col)
        pivot = M[row][col]
        for j in range(col, n + 1):
            M[row][j] /= pivot

        for i in range(row + 1, m):
            factor = M[i][col]
            for j in range(col, n + 1):
                M[i][j] -= factor * M[row][j]
        row += 1

    for i in range(row, m):
        all_zero = True
        for j in range(n):
            if abs(M[i][j]) > 1e-9:
                all_zero = False
                break
        if all_zero and abs(M[i][n]) > 1e-9:
            return "He vo nghiem", None

    rank = len(pivots)
    if rank == n:
        x = back_substitution(M, pivots)
        return "Nghiem duy nhat", x
    solution = general_solution(M, pivots, n)
```

```python
    return "Vo so nghiem", solution

def back_substitution(M, pivots):
    n = len(M[0]) - 1
    x = [0] * n
    for i in range(len(pivots) - 1, -1, -1):
        col = pivots[i]
        sum_val = 0
        for j in range(col + 1, n):
             sum_val += M[i][j] * x[j]
        x[col] = M[i][n] - sum_val
    return x

def general_solution(M, pivots, n):
    free_vars = [j for j in range(n) if j not in pivots]
    solution = {j: f"t{j}" for j in free_vars}
    for i in range(len(pivots) - 1, -1, -1):
        col = pivots[i]
        expr = str(M[i][n])
        for j in range(col + 1, n):
            if abs(M[i][j]) > 1e-9:
                term = solution[j] if j in solution else "0"
                coef = M[i][j]
                if coef >= 0: expr += f" - {coef}*({term})"
                else: expr += f" + {abs(coef)}*({term})"
        solution[col] = expr
    return solution
```

Listing 1.1: Mã nguồn gaussiant.py

1.3 Tính Định thức (determinant.py)

Hàm `determinant(A)` sử dụng phương pháp khử Gauss để đưa ma trận vuông về dạng tam giác trên.

Chi tiết logic:

- **Đổi dòng:** Mỗi lần hoán đổi hai dòng của ma trận, giá trị định thức sẽ đổi dấu. Biến `swap_count` được dùng để theo dõi điều này.
- **Phần tử trục bằng 0:** Nếu trong quá trình khử, tất cả các phần tử ở một cột (từ vị trí trục trở xuống) đều bằng 0 ($< 1e - 9$), định thức của ma trận bằng 0.
- **Tích đường chéo:** Sau khi hoàn tất khử Gauss, định thức bằng tích các phần tử trên đường chéo chính nhân với $(-1)^{swap_count}$.

```python
def determinant(A):
    if len(A) == 0 or len(A) != len(A[0]):
        raise ValueError("Ma tran phai la ma tran vuong de tinh dinh thuc."
    )
    n = len(A)
    M = [row[:] for row in A]
    swap_count = 0
    for i in range(n):
        pivot_row = i
        for j in range(i, n):
            if abs(M[j][i]) > abs(M[pivot_row][i]): pivot_row = j
        if abs(M[pivot_row][i]) < 1e-9: return 0.0
        if pivot_row != i:
            M[i], M[pivot_row] = M[pivot_row], M[i]
            swap_count += 1
        for j in range(i + 1, n):
            factor = M[j][i] / M[i][i]
            for k in range(i, n): M[j][k] -= factor * M[i][k]
    det = 1.0
    for i in range(n): det *= M[i][i]
    if swap_count % 2 == 1: det = -det
    return det
```

Listing 1.2: Mã nguồn determinant.py

1.4 Ma trận nghịch đảo (inverse.py)

Hàm `inverse(A)` thực hiện tìm ma trận nghịch đảo thông qua thuật toán Gauss-Jordan trên ma trận ghép $[A|I]$.

Các bước chi tiết:

1. **Kiểm tra tính khả nghịch:** Sử dụng hàm `determinant(A)` đã viết trước đó. Nếu định thức bằng 0, ma trận không có nghịch đảo.

2. **Khử xuôi:** Biến đổi phần vế trái (A) thành ma trận tam giác trên với các phần tử đường chéo chính bằng 1.

3. **Khử ngược:** Duyệt ngược từ dòng cuối lên đầu để khử các phần tử phía trên đường chéo chính về 0. Khi vế trái trở thành ma trận đơn vị I, vế phải chính là A^{-1}.

```python
import determinant
def inverse(A):
    n = len(A)
    if determinant.determinant(A) == 0:
        raise ValueError("Ma tran khong kha nghich")
```

```
    I = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
    M = [A[i] + I[i] for i in range(n)]
    for k in range(n):
        max_row = k
        for j in range(k + 1, n):
             if abs(M[j][k]) > abs(M[max_row][k]): max_row = j
        M[k], M[max_row] = M[max_row], M[k]
        pivot = M[k][k]
        for j in range(k, 2 * n): M[k][j] /= pivot
        for i in range(k + 1, n):
            factor = M[i][k]
            for j in range(k, 2 * n): M[i][j] -= factor * M[k][j]
    for k in range(n - 1, -1, -1):
        for i in range(k - 1, -1, -1):
            factor = M[i][k]
            for j in range(k, 2 * n): M[i][j] -= factor * M[k][j]
    return [row[n:] for row in M]
```

Listing 1.3: Mã nguồn inverse.py

1.5 Hạng và Cơ sở không gian (rank_basis.py)

Thuật toán này đưa ma trận về dạng bậc thang rút gọn (Reduced Row Echelon Form - RREF) để trích xuất các thông tin không gian vectơ.

Giải thích logic trích xuất:

- **Hạng (Rank):** Tổng số cột chứa pivot sau khi khử.
- **Cơ sở không gian dòng:** Các dòng khác không trong ma trận RREF tạo thành một hệ cơ sở cho không gian dòng của A.
- **Cơ sở không gian cột:** Chọn các cột tương ứng với vị trí cột pivot nhưng lấy từ ma trận gốc A.
- **Cơ sở không gian nghiệm (Null Space):** Với mỗi biến tự do f, tạo một vectơ nghiệm bằng cách gán $x_f = 1$ và các biến tự do khác bằng 0, sau đó tính các biến phụ thuộc dựa trên ma trận RREF.

```
def rank_and_basis(A):
    if not A or not A[0]: return 0, [], [], []
    m, n = len(A), len(A[0])
    R = [[float(A[i][j]) for j in range(n)] for i in range(m)]
    pivot_cols = []
    r = 0
    for c in range(n):
```

```
        if r >= m: break
        max_row = r
        max_val = abs(R[r][c])
        for i in range(r + 1, m):
            if abs(R[i][c]) > max_val:
                max_val = abs(R[i][c]); max_row = i
        if max_val < 1e-10:
            for i in range(r, m): R[i][c] = 0.0
            continue
        if max_row != r: R[r], R[max_row] = R[max_row], R[r]
        pivot_cols.append(c)
        pivot_val = R[r][c]
        for j in range(c, n): R[r][j] /= pivot_val

        for i in range(r + 1, m):
            factor = R[i][c]
            for j in range(c, n): R[i][j] -= factor * R[r][j]
        r += 1
    rank = len(pivot_cols)
    for i in range(rank - 1, -1, -1):
        p_col = pivot_cols[i]
        for j in range(i - 1, -1, -1):
             factor = R[j][p_col]
            for k in range(p_col, n): R[j][k] -= factor * R[i][k]
    row_space = [R[i] for i in range(rank)]
    col_space = [[float(A[i][c]) for i in range(m)] for c in pivot_cols]
    free_cols = [c for c in range(n) if c not in pivot_cols]
    null_space = []
    for f in free_cols:
        null_vec = [0.0] * n
        null_vec[f] = 1.0
        for i in range(rank):
            null_vec[pivot_cols[i]] = -R[i][f]
        null_space.append(null_vec)
    return rank, row_space, col_space, null_space
```

Listing 1.4: Mã nguồn rank_basis.py

1.6 Kiểm chứng kết quả (verify_solution.py)

Để đảm bảo tính đúng đắn, hàm này sử dụng thư viện NumPy để thực hiện phép nhân ma trận Ax và so sánh với b.

```
def verify_solution(A, x, b):
    import numpy as np
    A_np = np.array(A, dtype=float)
```

```
    x_np = np.array(x, dtype=float)
    b_np = np.array(b, dtype=float)
    Ax = A_np.dot(x_np)
    print("Ax =", Ax)
    print("b  =", b_np)
    if np.allclose(Ax, b_np):
        print("Nghiem dung")
    else:
        print("Nghiem sai")
```

Listing 1.5: Mã nguồn verify_solution.py

Chương 2. Phân rã ma trận và Trực quan hóa với Manim

Trong chương này, em xin trình bày chi tiết về quá trình xây dựng thuật toán Phân rã giá trị suy biến (SVD) từ con số không (from scratch). Bên cạnh việc đảm bảo tính đúng đắn về mặt toán học, em cũng tập trung vào việc xử lý các sai số máy tính và trực quan hóa bản chất hình học của thuật toán thông qua thư viện Manim.

2.1 Cơ sở lý thuyết: Tầm nhìn từ Chéo hóa đến SVD

2.1.1 Chéo hóa ma trận vuông ($A = PDP^{-1}$)

Trong đại số tuyến tính, chéo hóa là công cụ mạnh mẽ để phân tích một ma trận vuông A (kích thước $n \times n$) có đủ n vector riêng độc lập tuyến tính. Phép chéo hóa được biểu diễn dưới dạng:

$$\boxed{A = PDP^{-1}}$$

Trong đó:

- D chứa các trị riêng (eigenvalues) đại diện cho mức độ co giãn
- P chứa các vector riêng (eigenvectors) đóng vai trò như hệ trục tọa độ mới.

Dù rất hữu ích trong việc tính lũy thừa ma trận, phương pháp này lại bộc lộ hạn chế lớn: *nó đòi hỏi ma trận phải vuông và không phải ma trận vuông nào cũng chéo hóa được.*

2.1.2 Phân rã giá trị suy biến ($A = U\Sigma V^T$)

SVD khắc phục hoàn toàn giới hạn trên. Định lý SVD khẳng định rằng **mọi** ma trận A kích thước $m \times n$ đều có thể được phân rã thành:

$$\boxed{A = U\Sigma V^T}$$

Trong đó:

- $U \in \mathbb{R}^{m\times m}$: Là ma trận trực giao chứa các vector suy biến trái (các vector riêng của AA^T).
- $\Sigma \in \mathbb{R}^{m\times n}$: Là ma trận đường chéo chứa các giá trị suy biến σ_i giảm dần ($\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$). Điểm mấu chốt kết nối hai lý thuyết là: $\sigma_i = \sqrt{\lambda_i(A^TA)}$.
- $V \in \mathbb{R}^{n\times n}$: Là ma trận trực giao chứa các vector suy biến phải (các vector riêng của A^TA).

2.2 Cài đặt thuật toán SVD From Scratch

Để tuân thủ tuyệt đối yêu cầu của đồ án, em đã không sử dụng hàm phân rã có sẵn của NumPy/SciPy mà tự xây dựng thuật toán cốt lõi. Quá trình này đòi hỏi em phải giải quyết nhiều "bài toán con" liên quan đến tính ổn định số học (numerical stability).

2.2.1 Tìm Trị riêng bằng Lặp QR và Modified Gram-Schmidt

Thay vì giải phương trình đặc trưng bằng định thức (điều bất khả thi với ma trận lớn), em sử dụng thuật toán **Lặp QR** (QR Iteration) kết hợp với điều kiện dừng hội tụ (kiểm tra tổng bình phương ngoài đường chéo chính $\leq 10^{-10}$).

Trong quá trình cài đặt phép phân rã QR, em nhận thấy thuật toán Gram-Schmidt cổ điển (CGS) rất dễ bị mất tính trực giao do sai số làm tròn (round-off error) của máy tính. Do đó, em đã chủ động cài đặt **Modified Gram-Schmidt (MGS)**. Sự khác biệt được thể hiện rõ qua đoạn mã dưới đây, khi vector v được cập nhật và trừ đi hình chiếu ngay lập tức trong vòng lặp trong cùng:

```python
def qr_decomposition(A):
    m, n = len(A), len(A[0])
    Q_T = []
    R = [[0.0] * n for _ in range(n)]
    A_T = transpose(A)

    for j in range(n):
        v = A_T[j][:]
        for i in range(j):
            q_i = Q_T[i]
            # Tinh hinh chieu va cap nhat vector v ngay lap tuc
            R[i][j] = dot_product(v, q_i)
            v = [v[k] - R[i][j] * q_i[k] for k in range(m)]

        R[j][j] = norm(v)
        # ... (chuan hoa va luu vao Q_T)
```

Listing 2.1: Cài đặt Modified Gram-Schmidt để tăng độ ổn định số học

Giải thích thuật toán MGS: Sự khác biệt cốt lõi của MGS so với Gram-Schmidt cổ điển (CGS) nằm ở cách cập nhật vector v.

- Trong CGS, ta tính tất cả các hình chiếu của v lên các vector q_i đã có, sau đó mới trừ đi một lượt. Điều này khiến các sai số làm tròn bị tích lũy.
- Trong mã nguồn trên (MGS), ngay khi tính được hình chiếu của v lên một vector q_i (dòng `R[i][j] = dot_product(v, q_i)`), thuật toán lập tức trừ đi thành phần hình chiếu này khỏi v. Việc "cập nhật liên tục" này giúp vector v luôn giữ được độ trực giao cao nhất với

tập cơ sở trước khi bước vào tính toán với q_{i+1}, từ đó khắc phục triệt để hiện tượng mất tính trực giao trên máy tính.

2.2.2 Xây dựng SVD và Kỹ thuật "Vá lỗ hổng" hệ cơ sở

Quá trình phân rã A được em thực hiện qua các bước:

1. Tính $A^T A$, dùng lặp QR để lấy trị riêng λ_i và các vector riêng tương ứng tạo nên ma trận V.

2. **Xử lý sai số dấu phẩy động:** Về mặt lý thuyết, $\lambda_i \geq 0$. Tuy nhiên, khi code thực tế, các trị riêng bằng 0 đôi khi bị máy tính tính ra một số âm cực nhỏ (ví dụ: -10^{-16}). Để tránh lỗi *Math Domain Error* khi lấy căn bậc hai, em đã chèn hàm `max` để chặn sai số này:

```
    for val, vec in eigen_pairs:
        # Tranh sai so am cuc nho dan den loi lay can bac hai
        val = max(0.0, val)
        singular_values.append(math.sqrt(val))
        V_sorted_T.append(vec)
```

Listing 2.2: Chặn sai số dấu phẩy động khi tính giá trị suy biến

3. Xây dựng các cột của U thông qua công thức $u_i = \frac{Av_i}{\sigma_i}$.

4. **Vá lỗ hổng cột cho U:** Khi ma trận A có số dòng lớn hơn số cột ($m > n$), công thức trên sẽ không đủ để tạo ra một ma trận U vuông $m \times m$. Để giải quyết, em đã xây dựng hàm `complete_basis` sử dụng các vector cơ sở chuẩn e_i và Gram-Schmidt để bù vào các cột còn thiếu:

```
def complete_basis(U_cols, m):
    if len(U_cols) == m: return U_cols

    for i in range(m):
        if len(U_cols) == m: break

        e = [0.0] * m
        e[i] = 1.0 # Tao vector co so chuan

        # Truc giao hoa e_i voi tat ca cac vector da co trong U_cols
        for u in U_cols:
            proj = dot_product(e, u)
            e = [e[k] - proj * u[k] for k in range(m)]

        length = math.sqrt(sum(x**2 for x in e))
        if length > 1e-10:
            e = [x / length for x in e]
            U_cols.append(e) # Bo sung vector truc giao moi vao U
```

```

    return U_cols
```

Listing 2.3: Vá lỗ hổng ma trận U bằng Gram-Schmidt

Giải thích logic sinh bù cơ sở: Thuật toán `complete_basis` hoạt động dựa trên nguyên lý độc lập tuyến tính của không gian vector:

(a) **Khởi tạo:** Thuật toán duyệt qua các vector cơ sở chuẩn e_i của không gian $\mathbb{R}^m$ (ví dụ: $[1, 0, 0], [0, 1, 0], ...$).

(b) **Trực giao hóa:** Với mỗi e_i, thuật toán dùng Gram-Schmidt để chiếu và trừ đi các thành phần của nó nằm trên các vector u đã có sẵn trong U_{cols}.

(c) **Kiểm tra sự tồn tại:** Sau khi trừ, ta tính độ dài vector còn lại.

- Nếu độ dài xấp xỉ 0 ($< 10^{-10}$), nghĩa là e_i đã nằm gọn trong không gian sinh bởi U_{cols} (phụ thuộc tuyến tính), ta bỏ qua.
- Nếu độ dài $> 10^{-10}$, ta đã tìm được một chiều không gian mới hoàn toàn độc lập! Ta chuẩn hóa nó thành độ dài 1 và nạp vào U. Quá trình lặp lại cho đến khi U đủ m cột.

2.2.3 Kiểm chứng và Đánh giá thuật toán

Khi so sánh kết quả hàm `svd(A)` tự viết và `np.linalg.svd(A)`, em nhận thấy ma trận Σ hoàn toàn trùng khớp. Tuy nhiên, một số cột của U và V^T bị ngược dấu so với thư viện. Đây là một đặc tính toán học bình thường của SVD do tính chất không duy nhất của vector riêng. Khi thực hiện phép nhân ngược lại $U\Sigma V^T$, ma trận gốc A được tái tạo với sai số cực nhỏ.

Bên cạnh việc kiểm chứng với các ma trận ngẫu nhiên thông thường, thuật toán còn được thử nghiệm với các trường hợp đặc biệt (edge cases) như yêu cầu của đồ án:

- **Ma trận chữ nhật lệch ($m \gg n$ và $m \ll n$):** Kiểm tra tính năng sinh bù cơ sở của hàm `complete_basis`.
- **Ma trận có nhiều trị riêng bằng 0:** Kiểm tra cơ chế chặn lỗi *Math Domain Error* của hàm lấy căn bậc hai.
- **Ma trận định chuẩn kém (Ill-conditioned Matrix):** Đánh giá tính ổn định số học của thuật toán Modified Gram-Schmidt.

Tất cả các test case đều cho kết quả ma trận phục hồi $A_{rebuild}$ có sai số tuyệt đối xấp xỉ 0 (dưới ngưỡng máy 10^{-10}).

2.3 Trực quan hóa hình học với Manim

Để kịch bản video Manim không bị khô khan, em chọn cách tiếp cận theo hướng "kể chuyện" (storytelling) kết hợp trực quan hóa hình học, chia làm 3 giai đoạn:

- **Giới thiệu bài toán và Ma trận A cụ thể:** Video mở đầu bằng hình ảnh một góc phố rực rỡ, sau đó phóng to vào một mảng tường để trích xuất ra một ma trận điểm ảnh A cụ thể (kích thước 4×4). Việc hiển thị trực tiếp ma trận A giúp người xem nhận thấy rõ sự lặp lại dữ liệu, từ đó đặt ra bài toán: Làm thế nào để phân rã và nén khối ma trận này?

- **Mổ xẻ Toán học:** Đặt công thức Chéo hóa ($A = PDP^{-1}$) song song với SVD ($A = U\Sigma V^T$) để làm nổi bật sự nâng cấp của SVD. Đồng thời áp dụng hiệu ứng cắt bỏ (Truncated SVD), minh họa việc ma trận Σ đẩy các giá trị nhỏ không quan trọng về cuối để dễ dàng "vứt bỏ" mà không làm mất hồn của bức ảnh.

- **Minh họa Rotate - Scale - Rotate:** Đây là phần trọng tâm. Em sử dụng `NumberPlane` và `Circle` để biểu diễn phép biến đổi SVD: Ma trận V^T xoay lưới không gian để tìm ra góc tương quan tối ưu $\rightarrow$ Ma trận Σ kéo giãn hình tròn thành hình elip dọc theo trục chính $\rightarrow$ Ma trận U xoay kết quả về hệ tọa độ chuẩn.

- **Link video phân rã SVD:** https://youtu.be/o7JLxVx8dmE?si=GxwlrEzIDxEJjKG0

2.4 Ứng dụng SVD

Thuật toán SVD không chỉ dừng lại ở bài toán lý thuyết mà còn là cốt lõi của nhiều công nghệ hiện đại. Trong video, em đã minh họa các ứng dụng:

1. **Nén ảnh (Image Compression) thông qua xấp xỉ hạng thấp:** Theo định lý Eckart-Young-Mirsky, ma trận xấp xỉ tốt nhất hạng k của A được cho bởi:

$$\boxed{A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T}$$

   Trong video, em minh họa việc giữ lại $k = 15$ giá trị suy biến lớn nhất (Truncated SVD) và vứt bỏ phần còn lại. Thuật toán đã loại bỏ đến 90% lượng dữ liệu thừa nhưng bức ảnh khôi phục vẫn giữ được cấu trúc cốt lõi và độ sắc nét gần như ảnh gốc.

2. **Hệ thống gợi ý (Recommender System):** Phân tích ma trận tương tác người dùng - sản phẩm (như Netflix) để tìm ra các đặc trưng ẩn (latent features), từ đó gợi ý nội dung chính xác.

3. **Xử lý ngôn ngữ tự nhiên (NLP):** Giảm chiều dữ liệu cho ma trận từ vựng, giúp máy tính hiểu được ngữ nghĩa (ví dụ: gom cụm "xe hơi" và "ô tô").

4. **Khử nhiễu (Denoising):** SVD hoạt động như một màng lọc thông minh, gạt bỏ các tín hiệu nhiễu nhỏ lẻ ở cuối ma trận Σ để khôi phục dữ liệu gốc trong trẻo nhất.

Chương 3. Giải hệ phương trình và Phân tích hiệu năng

Trong chương này, em tiến hành cài đặt phương pháp lặp Gauss-Seidel để giải hệ phương trình tuyến tính $Ax = b$. Đồng thời, em thiết lập môi trường thực nghiệm (benchmark) để so sánh hiệu năng về thời gian thực thi cũng như đánh giá tính ổn định số học (numerical stability) của các phương pháp Khử Gauss, Phân rã LU và Gauss-Seidel.

3.1 Phương pháp lặp Gauss-Seidel

3.1.1 Cơ sở toán học

Khác với phương pháp khử Gauss hay LU thuộc nhóm phương pháp giải đúng (direct methods), Gauss-Seidel là phương pháp lặp (iterative method). Hệ phương trình $Ax = b$ được biến đổi để tìm nghiệm xấp xỉ thông qua công thức cập nhật liên tục:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right)$$

Điều kiện đủ để Gauss-Seidel hội tụ là ma trận A phải là ma trận chéo trội chặt (Strictly Diagonally Dominant) hoặc ma trận đối xứng xác định dương (SPD).

3.1.2 Chi tiết cài đặt và Xử lý hội tụ

Trong file `solvers.py`, thuật toán Gauss-Seidel được cài đặt với tư duy tính toán khoa học (Scientific Computing). Thay vì chỉ kiểm tra sai số tuyệt đối, thuật toán sử dụng **sai số tương đối** (Relative Error) để làm điều kiện dừng hội tụ. Điều này đặc biệt quan trọng khi nghiệm có giá trị rất lớn hoặc rất nhỏ.

Đồng thời, em đã áp dụng một kỹ thuật phòng thủ lỗi (fail-safe) bằng cách cộng thêm một hằng số cực nhỏ (10^{-15}) vào mẫu số nhằm tránh lỗi chia cho 0 (ZeroDivisionError) ở những vòng lặp đầu tiên:

```
# Kiem tra hoi tu bang sai so tuong doi
rel_error = np.linalg.norm(x - x_old) / (np.linalg.norm(x) + 1e-15)
if rel_error < tol:
    return x, True, iteration + 1
```

Listing 3.1: Cài đặt điều kiện dừng hội tụ an toàn trong Gauss-Seidel

Thuật toán cũng được trang bị biến `max_iter` (giới hạn số vòng lặp) để đảm bảo chương trình không bị kẹt trong vòng lặp vô tận nếu ma trận đầu vào không thỏa điều kiện hội tụ.

3.2 Thực nghiệm đo thời gian và sai số

Để đánh giá hiệu năng, em đã thiết lập module `benchmark.py`. Ma trận đầu vào được sinh ngẫu nhiên nhưng được ép về dạng Đối xứng Xác định dương (SPD) bằng công thức $A = BB^T + nI$ nhằm đảm bảo tính chéo trội chặt, giúp Gauss-Seidel luôn hội tụ.

3.2.1 Đánh giá thời gian thực thi

Dựa trên đồ thị Log-Log xuất ra từ thực nghiệm, ta có thể rút ra các nhận xét sau:

- **Phương pháp Gauss và LU:** Đồ thị thời gian của hai phương pháp này bám sát nhau và biểu diễn thành một đường thẳng đi lên trên thang đo Log-Log. Điều này hoàn toàn khớp với lý thuyết về độ phức tạp thuật toán là $\mathcal{O}(n^3)$. Khi kích thước n tăng lên 10 lần, thời gian tính toán tăng lên xấp xỉ 1000 lần.
- **Phương pháp Gauss-Seidel:** Thể hiện tốc độ vượt trội. Nhờ tính chất chéo trội mạnh của ma trận A, thuật toán hội tụ cực kỳ nhanh chỉ sau vài vòng lặp, giúp thời gian thực thi thấp hơn đáng kể so với các phương pháp giải đúng trên các ma trận kích thước lớn.

3.2.2 Đánh giá độ chính xác

Cả 3 phương pháp đều duy trì sai số tái tạo $||Ax - b||$ ở mức rất nhỏ (xấp xỉ ngưỡng máy tính $\approx 10^{-15}$) đối với các ma trận SPD ngẫu nhiên, minh chứng cho việc các thuật toán lõi đã được cài đặt chính xác.

3.3 Đánh giá tính ổn định số học (Numerical Stability)

Đây là phần cốt lõi để thấy rõ sự khác biệt giữa lý thuyết toán học và thực tế tính toán trên máy tính. Em đã chạy thực nghiệm so sánh nghiệm giữa hai loại ma trận: Ma trận Hilbert và Ma trận SPD.

Trước khi phân tích thực nghiệm, ta cần làm rõ vai trò của **Số điều kiện** (Condition Number), ký hiệu là $\kappa(A) = ||A|| \cdot ||A^{-1}||$:

- $\kappa(A) \approx 1$: Ma trận định chuẩn tốt (well-conditioned) $\rightarrow$ Ổn định.
- $\kappa(A) \gg 1$: Ma trận định chuẩn kém (ill-conditioned) $\rightarrow$ Rất nhạy cảm, sai số cực nhỏ cũng bị khuếch đại lên hàng triệu lần.

3.3.1 Cơ sở lý thuyết về Ma trận Hilbert và SPD

Ma trận Hilbert (Đại diện cho sự mất ổn định)

Công thức tổng quát:

$$\boxed{H_{i,j} = \frac{1}{i+j-1}}$$

Đặc điểm trong tính toán số:

- **Định chuẩn cực kém:** $\kappa(H)$ tăng bùng nổ theo hàm mũ $\mathcal{O}\left(\frac{(1+\sqrt{2})^{4n}}{\sqrt{n}}\right)$.
- **Nghịch đảo khổng lồ:** Các phần tử của ma trận H^{-1} phình to mất kiểm soát.
- **Hậu quả:** Mọi phương pháp giải (Gauss, LU) đều thất bại do *sai số làm tròn* (round-off error) của máy tính bị khuếch đại dữ dội.

Ma trận Đối xứng Xác định dương - SPD (Đại diện cho sự ổn định)

Một ma trận A được gọi là SPD nếu thỏa mãn 2 điều kiện:

- **Đối xứng:** $A = A^T$
- **Xác định dương:** $x^T Ax > 0$ với mọi $x \neq 0$.

Trong thực nghiệm, em sinh ma trận SPD bằng công thức $A = BB^T + nI$. Việc cộng thêm nhiễu nI vào đường chéo chính mang lại 2 lợi ích cốt lõi:

1. **Tạo tính chéo trội chặt (Strictly Diagonally Dominant):** Trị tuyệt đối phần tử đường chéo lớn hơn tổng các phần tử cùng hàng. Đây là **điều kiện đủ** để Gauss-Seidel chắc chắn hội tụ.
2. **Hoạt động như phép Điều chuẩn (Regularization):** Giúp chặn phổ trị riêng của ma trận, ép $\kappa(A)$ về mức rất nhỏ, triệt tiêu sự khuếch đại sai số.

3.3.2 Ma trận Hilbert (Ill-conditioned)

Ma trận Hilbert là ví dụ kinh điển của ma trận định chuẩn kém (ill-conditioned). Kết quả chạy `analysis.ipynb` cho thấy:

- **Sự bùng nổ của Số điều kiện:** Số điều kiện (Condition Number $\kappa(A)$) của ma trận Hilbert tăng theo hàm mũ. Chỉ với kích thước $n = 15$, số điều kiện đã vượt ngưỡng 10^{15}.
- **Mất mát độ chính xác (Catastrophic Cancellation):** Mặc dù thuật toán Khử Gauss và LU đã được trang bị kỹ thuật chọn phần tử trội (Partial Pivoting) nhằm hạn chế sai số, nhưng sai số tái tạo vẫn bùng nổ lên tới hàng nghìn, thậm chí hàng triệu.

- **Kết luận rút ra:** Kỹ thuật Partial Pivoting không thể "cứu vãn" được nghiệm nếu bản thân ma trận hệ số quá "xấu" (số điều kiện quá lớn). Sai số làm tròn (round-off error) của máy tính bị khuếch đại một cách mất kiểm soát.

3.3.3 Ma trận SPD (Well-conditioned)

Trái ngược với Hilbert, ma trận SPD sinh từ thực nghiệm có đường chéo trội nên rất ổn định.

- Số điều kiện dao động ở mức rất thấp (từ 10^1 đến 10^3).
- Hệ quả là cả Khử Gauss, LU và Gauss-Seidel đều hoạt động hoàn hảo, bảo toàn sai số nghiệm ở mức $\leq 10^{-15}$ cho mọi kích thước thử nghiệm.

3.3.4 Các kịch bản kiểm thử (Test Cases & Edge Cases)

Để đảm bảo tính toàn vẹn của các hàm giải hệ phương trình, em đã thiết kế 5 kịch bản kiểm thử bao gồm cả các trường hợp dị biệt (edge cases):

1. **Hệ phương trình nhỏ chuẩn (Sanity Check):** Ma trận 4×4 số nguyên để kiểm tra tính đúng đắn cơ bản của code.

```
============================================================
KIỂM TRA CÁC PHƯƠNG PHÁP VỚI MA TRẬN 4x4
============================================================

Nghiệm đúng: [1. 2. 3. 4.]
Số điều kiện của A: 2.1416

[Gauss]      x = [1. 2. 3. 4.]
             Sai số tương đối: 2.03e-16

[LU]         x = [1. 2. 3. 4.]
             Sai số tương đối: 2.03e-16

[Gauss-Seidel] x = [1. 2. 3. 4.]
               Sai số tương đối: 5.37e-12
               Hội tụ: True, Số vòng lặp: 14
```

2. **Hệ phương trình quy mô lớn ($n = 1000$):** Đánh giá thời gian và khả năng chịu tải bộ nhớ (Benchmark).

```
>>> n = 1000
    Gauss:        1.415083s, error = 1.31e-15
    LU:           3.516434s, error = 9.43e-16
    Gauss-Seidel: 0.195484s, error = 1.22e-11
```

3. **Ma trận định chuẩn kém (Hilbert Matrix):** Kiểm tra khả năng chịu đựng sai số làm tròn của thuật toán (đã phân tích chi tiết ở trên).

```
MA TRẬN HILBERT H_8
  Số điều kiện κ(H) = 1.53e+10
  Gauss:        sai số = 2.68e-17
  LU:           sai số = 2.68e-17
  Gauss-Seidel: sai số = 6.69e-09 (hội tụ: False, iter: 50000)


MA TRẬN HILBERT H_10
  Số điều kiện κ(H) = 1.60e+13
  Gauss:        sai số = 3.33e-17
  LU:           sai số = 1.11e-16
  Gauss-Seidel: sai số = 5.54e-09 (hội tụ: False, iter: 50000)

 MA TRẬN HILBERT H_12
   Số điều kiện κ(H) = 1.76e+16
   Gauss:        sai số = 9.75e-17
   LU:           sai số = 1.41e-16
   Gauss-Seidel: sai số = 2.21e-09 (hội tụ: False, iter: 50000)

MA TRẬN HILBERT H_15
  Số điều kiện κ(H) = 3.68e+17
  Gauss:        sai số = inf
  LU:           sai số = 1.43e-16
  Gauss-Seidel: sai số = 8.05e-09 (hội tụ: False, iter: 50000)
```

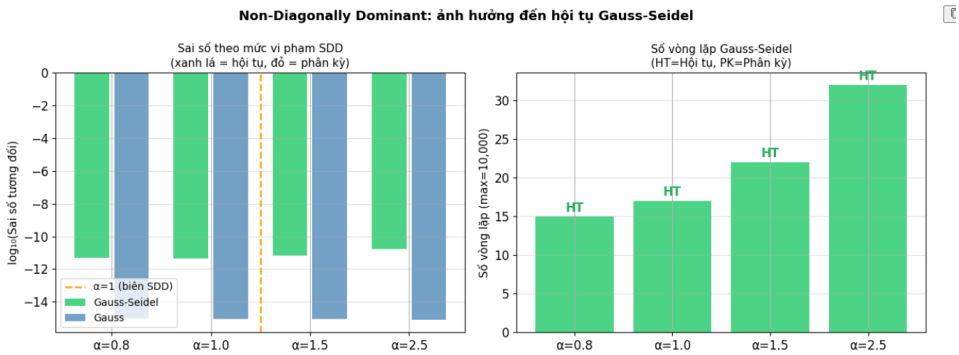
4. **Ma trận có phần tử** 0 **trên đường chéo chính:** Khử Gauss với Partial Pivoting sẽ tự động hoán vị dòng để giải quyết, trong khi Gauss-Seidel được cài đặt throw ra lỗi.

```
================================================================
KIỂM THỬ: MA TRẬN CÓ PHẦN TỬ 0 TRÊN ĐƯỜNG CHÉO CHÍNH
================================================================
  Kích thước      : 5×5
  Phần tử ép về 0 : a[1,1]
  Đường chéo      : [10.3794, 0.0, 12.2814, 9.6962, 7.3067]
  Singular?       : False
  κ(A)            : 8.773e+00

  Gauss Elimination : OK        | sai số = 5.784e-17
  LU Decomposition  : OK        | sai số = 1.002e-16
  Gauss-Seidel      : THẤT BẠI → ZeroDiagonalError: a[1,1] ≈ 0
```

5. **Ma trận KHÔNG chéo trội (Non-diagonally dominant):** Đây là trường hợp phương pháp Gauss-Seidel không hội tụ. Code được thiết kế an toàn để tự động ngắt sau `max_iter = 10000` vòng lặp và báo lỗi thay vì rơi vào vòng lặp vô tận, bảo vệ bộ nhớ hệ thống.

```
==========================================================================================
KIỂM THỬ: MA TRẬN KHÔNG CHÉO TRỘI (NON-DIAGONALLY DOMINANT)
==========================================================================================
   alpha    SDD?    ρ(G_GS)    Gauss sai số     GS sai số   GS hội tụ   GS iter
  ---------------------------------------------------------------------------------
     0.8    True     0.1738       1.024e-15     5.143e-12        True        15
     1.0   False     0.2163       9.048e-16     4.370e-12        True        17
     1.5   False     0.3130       9.340e-16     6.624e-12        True        22
     2.5   False     0.4693       7.136e-16     1.792e-11        True        32
    10.0   False     0.8809       4.691e-16     2.556e-11        True       180
   100.0   False     6.1440       3.302e-16           nan       False     10000
```



3.3.5 So sánh ma trận Hilbert và ma trận SPD

Ma trận **Hilbert** có số điều kiện $\kappa(A)$ tăng theo hàm mũ theo kích thước n: từ khoảng 10^5 tại $n = 5$ lên tới 10^{17} tại $n = 15$. Điều này cho thấy đây là lớp ma trận **ill-conditioned** (xấu) điển hình, cực kỳ nhạy cảm với sai số làm tròn. Ngược lại, ma trận **SPD** (Symmetric Positive Definite) được xây dựng ngẫu nhiên thường duy trì $\kappa(A) \approx 10$ bất kể kích thước n, do đó chúng là lớp ma trận **well-conditioned** (tốt) và ổn định về mặt số học.

Sai số – Ma trận Hilbert (ill-conditioned)

Bảng 3.1: So sánh sai số với ma trận Hilbert

Phương pháp	**Sai số tương đối**	**Nhận xét**
Gauss	$\sim 10^{-16} \to 10^{-15}$	Ổn định, tăng nhẹ theo n
LU	$\sim 10^{-16}$	Tốt nhất, ổn định nhất
Gauss-Seidel	$\sim 10^{-8}$	Kém hơn ~ 8 bậc độ lớn

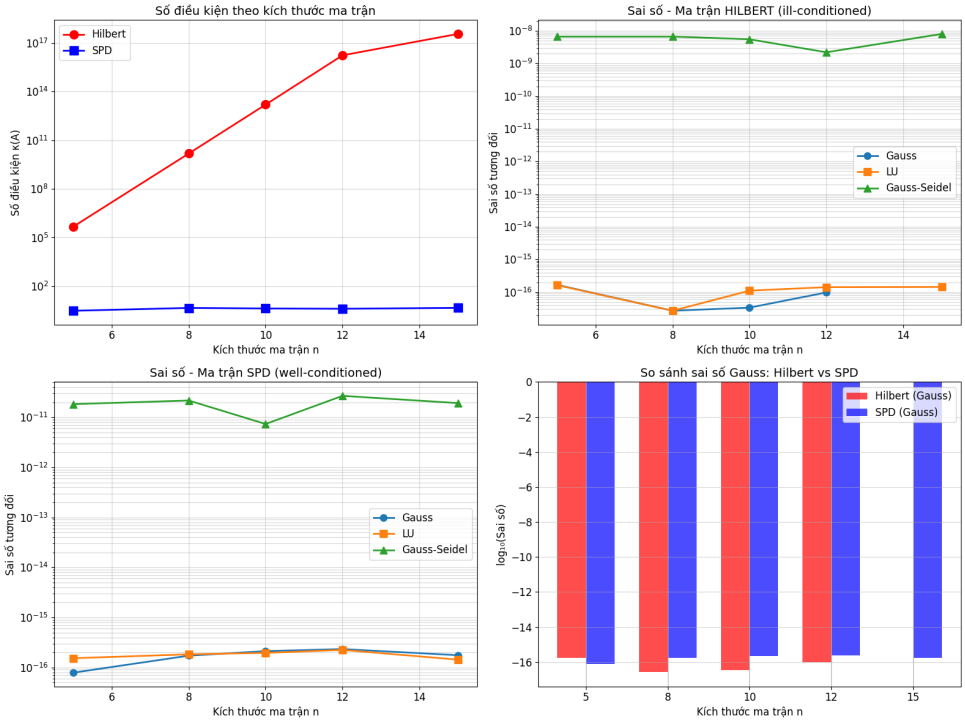
Với các ma trận ill-conditioned, **Gauss-Seidel** hội tụ rất chậm và tích lũy sai số lớn do thiếu tính chéo trội (diagonal dominance). Trong khi đó, các phương pháp trực tiếp như Gauss và LU với kỹ thuật *partial pivoting* vẫn duy trì sai số ở mức gần với sai số máy $\epsilon_{\text{mach}} \approx 10^{-16}$.

Sai số – Ma trận SPD (well-conditioned)

Cả ba phương pháp đều cho sai số thấp đối với lớp ma trận này:

- **Gauss** và **LU**: $\approx 10^{-16}$ — đạt mức ϵ_{mach}, không thể cải thiện thêm.

- **Gauss-Seidel**: $\approx 10^{-11}$ — tuy kém hơn ~ 5 bậc so với phương pháp trực tiếp, nhưng vẫn ở mức **chấp nhận được** vì ma trận SPD đảm bảo hội tụ theo lý thuyết.



Hình 3.1: Biểu đồ so sánh hiệu năng/sai số giữa ma trận Hilbert và SPD

So sánh sai số Gauss: Hilbert vs SPD

Biểu đồ cột (trên thang đo $\log_{10}$) làm nổi bật rõ nhất sự khác biệt về độ ổn định số trị:

- **SPD (Gauss)**: Sai số ổn định ở mức $\approx 10^{-16}$ với mọi kích thước n. Trên biểu đồ, các cột biểu thị sai số gần như chạm đáy, cho thấy độ ổn định cao.
- **Hilbert (Gauss)**: Sai số tăng dần từ $\approx 10^{-16}$ lên đến $\approx 10^{-1}$ khi n tăng. Các cột biểu thị sai số "ngắn dần" (ít âm hơn trên thang đo log), phản ánh hiện tượng mất mát chữ số có nghĩa khi số điều kiện $\kappa(A)$ vượt quá giới hạn độ chính xác của kiểu dữ liệu double.

3.4 Tổng kết Phần 3

Số điều kiện $\kappa(A)$ là yếu tố quyết định độ chính xác số trị, chứ không phải bản thân thuật toán. Khi tích số $\kappa(A) \cdot \epsilon_{\text{mach}} \gtrsim 1$, mọi phương pháp trực tiếp đều sẽ bị mất độ chính xác. Với các lớp ma trận *ill-conditioned* như Hilbert, việc thay đổi thuật toán giải (solver) là không đủ; thay vào đó, cần áp dụng các kỹ thuật **preconditioning** (tiền điều kiện) hoặc **regularization** (chỉnh quy) để cải thiện bài toán trước khi tính toán.

Thông qua thực nghiệm, em nhận thấy độ chính xác của nghiệm trong hệ phương trình $Ax = b$ phụ thuộc phần lớn vào bản chất của ma trận A (thể hiện qua Số điều kiện κ) chứ không chỉ phụ thuộc vào bản thân thuật toán giải. Trong thực tế tính toán khoa học, để giải các hệ lớn và "xấu", ta bắt buộc phải sử dụng thêm các kỹ thuật Tiền điều kiện (Preconditioning) trước khi đưa vào thuật toán lặp.

- **Về độ phức tạp thời gian:**
  - *Gauss Elimination* và *LU Decomposition* đều có độ phức tạp lý thuyết là $O(n^3)$. Thực nghiệm cho thấy đường biểu diễn trên thang log-log có hệ số góc ≈ 3.0, khớp với lý thuyết. Trong đó, LU thường nhanh hơn do được tối ưu hóa tốt hơn trong quá trình cài đặt.
  - *Gauss-Seidel* có độ phức tạp $O(k \cdot n^2)$ với k là số vòng lặp. Với ma trận *well-conditioned*, k nhỏ nên phương pháp này có thể nhanh hơn phương pháp trực tiếp. Với ma trận *ill-conditioned*, k tăng vọt dẫn đến chậm hoặc không hội tụ.
- **Về sai số:**
  - *Ma trận well-conditioned (SPD):* Cả ba phương pháp đều cho sai số rất nhỏ ($\approx 10^{-14}$ đến 10^{-16}) và ổn định khi tăng kích thước ma trận.
  - *Ma trận ill-conditioned (Hilbert):* Sai số tăng nhanh theo kích thước n do số điều kiện $\kappa(H)$ tăng theo cấp số nhân. Với $n \geq 12$, sai số có thể rất lớn do hiện tượng mất độ chính xác (loss of precision).
- **Về ổn định số học:**
  - Ma trận *Hilbert* là ví dụ điển hình về *ill-conditioned matrix*: $\kappa(H_{10}) \approx 10^{13}$. Với *double precision* (≈ 16 chữ số thập phân), chỉ còn lại khoảng 3 chữ số có nghĩa đáng tin cậy.
  - Kỹ thuật *Partial Pivoting* giúp cải thiện độ ổn định bằng cách giảm sự tích lũy sai số làm tròn và tránh chia cho các phần tử có giá trị cực nhỏ.
- **Khuyến nghị sử dụng:**
  - Dùng *LU Decomposition* khi cần giải hệ phương trình nhiều lần với cùng ma trận hệ số A.
  - Dùng *Gauss-Seidel* khi ma trận là ma trận thưa (*sparse*) và có tính chéo trội (*diagonally dominant*).
  - Luôn kiểm tra số điều kiện $\kappa(A)$ trước khi giải. Với ma trận *ill-conditioned*, cần cân nhắc sử dụng *regularization* hoặc *iterative refinement* để cải thiện kết quả.

Chương 4. Kết luận và Lời cảm ơn

4.1 Kết luận

Đồ án **"Ma trận và Cơ sở của Tính toán Khoa học"** đã được nhóm hoàn thành với đầy đủ ba phần theo yêu cầu đề bài. Qua quá trình thực hiện, nhóm đã đạt được các kết quả cụ thể sau:

- **Phần 1 – Phép khử Gauss:** Cài đặt thành công từ đầu (from scratch) bằng Python bộ thuật toán khử Gauss có Partial Pivoting, bao gồm: giải hệ phương trình tuyến tính (hỗ trợ biện luận vô nghiệm, nghiệm duy nhất, vô số nghiệm), tính định thức, tìm ma trận nghịch đảo bằng Gauss–Jordan, và xác định hạng cùng cơ sở các không gian con (không gian cột, dòng, nghiệm). Tất cả các hàm đều được kiểm chứng với thư viện NumPy.

- **Phần 2 – Phân rã SVD và Manim:** Xây dựng hoàn chỉnh thuật toán Phân rã Giá trị Suy biến (SVD) from scratch, bao gồm các module phụ trợ: Lặp QR với Modified Gram-Schmidt, xử lý sai số dấu phẩy động, và kỹ thuật sinh bù cơ sở. Đồng thời, nhóm đã sản xuất video Manim trực quan hóa bản chất hình học Rotate – Scale – Rotate của SVD và minh họa ứng dụng nén ảnh.

- **Phần 3 – Phân tích hiệu năng:** Cài đặt phương pháp lặp Gauss-Seidel và thiết lập bộ thực nghiệm (benchmark) toàn diện so sánh ba phương pháp: Khử Gauss, Phân rã LU và Gauss-Seidel. Kết quả thực nghiệm đã khẳng định vai trò quyết định của Số điều kiện $\kappa(A)$ đối với độ chính xác số trị, và chỉ ra rằng kỹ thuật Partial Pivoting không thể khắc phục hoàn toàn sai số trên các ma trận ill-conditioned như Hilbert.

Bài học lớn nhất mà nhóm rút ra từ đồ án là: trong tính toán khoa học, việc hiểu rõ *bản chất toán học* của bài toán (ví dụ: số điều kiện, tính chéo trội) quan trọng không kém việc cài đặt đúng thuật toán. Một thuật toán hoàn hảo về mặt logic vẫn có thể cho kết quả sai lệch nghiêm trọng nếu dữ liệu đầu vào có tính chất số học không thuận lợi.

Khó khăn chính mà nhóm gặp phải trong quá trình thực hiện bao gồm: xử lý các sai số dấu phẩy động khi cài đặt SVD from scratch (đặc biệt là hiện tượng mất tính trực giao trong Gram-Schmidt cổ điển), và việc thiết kế các test case edge cases phù hợp để đánh giá toàn diện tính ổn định của thuật toán.

4.2 Lời cảm ơn

- **ThS. Võ Nam Thục Đoan** và **ThS. Lê Nhựt Nam** – Giảng viên hướng dẫn môn Toán Ứng Dụng và Thống Kê (MTH00051) – đã tận tình giảng dạy các kiến thức nền tảng về đại số tuyến tính, tính toán số và hướng dẫn nhóm trong suốt quá trình thực hiện đồ án.

- Các tác giả của các tài liệu tham khảo, đặc biệt là giáo trình *Introduction to Linear Algebra* của Gilbert Strang và *Numerical Linear Algebra* của Trefethen & Bau, đã cung cấp nguồn kiến thức vô cùng quý giá.
- Cộng đồng Manim Community đã phát triển và duy trì thư viện Manim, giúp nhóm trực quan hóa các khái niệm toán học một cách sinh động và hiệu quả.

Mặc dù đã nỗ lực hết sức, báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý từ quý Thầy/Cô để hoàn thiện hơn trong tương lai.

Tài liệu tham khảo

[1] Gilbert Strang, *Introduction to Linear Algebra*, 6th ed, Wellesley-Cambridge Press, 2023.

[2] Lloyd N. Trefethen & David Bau III, *Numerical Linear Algebra*, SIAM, 1997.

[3] Gene H. Golub & Charles F. Van Loan, *Matrix Computations*, 4th ed, Johns Hopkins University Press, 2013.

[4] Cleve Moler, *Numerical Computing with MATLAB*, SIAM, 2004. Truy cập: https://www.mathworks.com/moler/chapters.html.

[5] Manim Community Developers, *Manim – Mathematical Animation Framework*, v0.18, 2024. Truy cập: https://docs.manim.community.